Handover — i18n corrections workflow (2026-04-30)

How to fix translation mistakes after L1–L3 ship. There are **two layers** with separate correction paths; a fix in the wrong place either won't propagate or won't survive a re-translation pass.

This doc replaces ad-hoc instructions in the L2 / L3 PR descriptions.

Layer 1 — UI catalog strings

What lives here: every UI string the platform renders that doesn't come out of the database. Page titles, button labels, form fields, empty-state copy, error messages, footer headings, IUCN category labels, etc.

Where they live: `frontend/messages/{en,fr,de,es}.json` — 801 keys × 4 locales, key parity enforced by `pnpm i18n:check`.

English (`en.json`) is the source of truth. Other locales translate each key's value; structure stays byte-identical.

Single-string fix

Find the key by searching the file for the visible English. Edit the value in the affected locale. From `frontend/`:

```
pnpm i18n:check    # confirm parity (still 801 keys × 4 locales)
pnpm dev           # browser-verify on /fr/<page>
```

Commit on a `fix/i18n-<topic>` branch, PR, merge.

Example

If "Comportement grégaire" reads wrong on the husbandry page:

```
   "behavior": {
-    "schooling": "Comportement grégaire",
+    "schooling": "Vie en banc",
```

Batched fixes

If a colleague returns a list of 30 corrections, drop them into one PR. For systematic terminology fixes (e.g., a glossary change that affects many strings), invoke `@conservation-writer` against the catalog with the correction policy spelled out.

Re-running MT from scratch

Rare. Only useful if DeepL fixed a known weakness and we want to take advantage of it. Re-running drops every writer-reviewed nuance:

```
cd frontend
pnpm i18n:translate fr --all   # retranslates every key
# Re-invoke @conservation-writer voice review on the diff.
```

When the English source itself changes

Adding a new key:

1. Add it to `en.json` with the English value.
2. **Also add it to `fr.json`, `de.json`, `es.json`** — initially with the English value as a placeholder. `pnpm i18n:check` fails the build otherwise.
3. Run `pnpm i18n:translate <locale>` for the locales that need it, or hand-translate the new key.

Editing an existing key's English value:

- The frontend renders the locale catalog regardless. If you fix `en.json` for an existing key, `fr.json` still shows the old French for that key. Re-translate that key (manually or via `pnpm i18n:translate fr --all` scoped narrowly).

Layer 2 — Species and Taxon content (database columns)

What lives here: species long-form prose (`description` , `ecology_notes` , `distribution_narrative` , `morphology`) and `Taxon.common_family_name` . These are `django-modeltranslation` columns: `description_en` , `description_fr` , `description_de` , `description_es` , etc. Same pattern for the other fields.

Pipeline: `mt_draft → writer_reviewed → human_approved → published` . Tracked in `i18n.TranslationStatus` rows (one per `{model, object_id, field, locale}` tuple).

Public-site gating: when `I18N_ENFORCE_REVIEW_GATE=True` in env, only `human_approved` (or `published`) translations reach public API payloads. Everything else falls back to English with the `(English)` badge on the frontend.

Single typo fix on an already-translated species

Cleanest path is the **TranslationStatus admin page**, which now lets you edit the translation inline:

1. Go to **Django admin → Translation pipeline → Translation statuses**.
2. Filter to the species: pick `locale` , `content_type=Species` , and search for the species id or scientific name in `notes` (or click the row from a list filter).
3. Click into the row's change page. You'll see two side-by-side fields:
   - **English source** (read-only)
   - **Target translation** (editable, pre-filled with the current French text)
4. Edit `Target translation` , click **Save**. The edit writes back to the underlying `Species.description_fr` column; the `post_save` signal keeps the TranslationStatus row in sync.

The status **does not auto-demote** on direct French edits. For typos, this is what you want — a one-character fix shouldn't kick a translation back through writer-review and human approval.

Substantive correction (more than a typo)

After editing the field as above, **manually demote** the status:

1. Back in the TranslationStatus list view, select the row(s) you just edited.
2. **Action** dropdown → **"Send back to mt_draft"** → **Go**.
3. The translation content is preserved; only the pipeline status resets. Reviewer (you / colleague) re-reads, then advances:

- **"Advance: mt_draft → writer_reviewed"**
- **"Approve: writer_reviewed → human_approved"** (stamps `human_approved_by` + `human_approved_at`).

English source changed → all locale translations auto-stale

Automatic. If you edit `description_en` (English source) on a species whose French was approved, the `post_save` signal demotes the French row back to `mt_draft` with a note:

> *"English source changed after this translation was approved; re-review needed."*

The French content is unchanged in the DB — it's still there, but the public site (with `I18N_ENFORCE_REVIEW_GATE=true`) serves English fallback until the row is re-approved.

Bulk correction across many species

Example: conservation-writer says "every occurrence of `registre généalogique` should be `livre généalogique`":

```
docker compose exec web python manage.py shell
```

```
from species.models import Species
from i18n.models import TranslationStatus
from django.contrib.contenttypes.models import ContentType

ct = ContentType.objects.get_for_model(Species)
TRANSLATABLE = ('description', 'ecology_notes', 'distribution_narrative',
'morphology')

for sp in Species.objects.iterator():
    changed = []
    for field in TRANSLATABLE:
        col = f"{field}_fr"
        current = getattr(sp, col, '') or ''
        if 'registre généalogique' in current:
            setattr(sp, col, current.replace('registre généalogique', 'livre
généalogique'))
            changed.append(col)
        if 'Registre généalogique' in current:
            setattr(sp, col, getattr(sp, col).replace('Registre généalogique',
'Livre généalogique'))
            if col not in changed: changed.append(col)
    if changed:
        sp.save(update_fields=changed)

# Bulk-demote affected rows so the change goes through review again.
# (Direct _fr edits don't auto-demote; substantive bulk edits do.)
TranslationStatus.objects.filter(
    content_type=ct, locale='fr',
    field__in=TRANSLATABLE,
```

```
    status__in=['writer_reviewed', 'human_approved'],
).update(status=TranslationStatus.Status.MT_DRAFT)
```

When a translation needs to be redone from scratch

Delete the locale column and re-run the MT pipeline:

```
# Targeted: clear French description for one species, force re-MT
sp = Species.objects.get(pk=42)
sp.description_fr = ''
sp.save(update_fields=['description_fr'])
TranslationStatus.objects.filter(
    content_type=ct, object_id=sp.pk, field='description', locale='fr',
).delete()
```

```
docker compose exec web python manage.py translate_species --locale fr --species 42
```

The MT pipeline re-translates, writes `description_fr`, and creates a fresh `TranslationStatus(status='mt_draft')` row.

For a whole family (`@conservation-writer` finishes a batched review with many corrections):

```
docker compose exec web python manage.py translate_species --locale fr --family
Bedotiidae --force
```

`--force` retranslates even rows whose target column is populated. Use with care — overwrites human-approved content.

Glossary updates

Both layers consult the locked-term glossary in `.claude/agents/conservation-writer.md`. When the platform owner changes a term standard (e.g., "studbook" → "livre généalogique" not "registre généalogique" — 2026-04-30), do all three:

1. Edit the glossary table in `.claude/agents/conservation-writer.md`.
2. Apply a one-shot fix-up to the existing French catalog and DB:

   ```
   # Layer 1 — catalog
   sed -i '' 's/registre généalogique/livre généalogique/g' \
     frontend/messages/fr.json

   # Layer 2 — DB (in Django shell, with bulk-demote afterward).
   ```

3. Future translations through `@conservation-writer` will respect the new convention without manual prompting.

Two edge cases worth knowing

1. **Editing a locale field directly does NOT auto-demote the `TranslationStatus` row.** The signal only auto-demotes on *English-source* changes (those genuinely invalidate the translation). Direct locale edits assume the editor knows what they're doing — substantive changes deserve a manual "Send back to mt_draft"; typos don't.
2. **The `(English)` fallback badge on the frontend** appears when `<field>_locale_actual !== <active_locale>` — meaning the gate rejected the locale content (either because it's empty, or because `I18N_ENFORCE_REVIEW_GATE=true` and the row isn't `human_approved`). If a reader sees the badge with English copy on a French URL, the system is working as designed: the French exists but isn't approved yet, OR `description_en` was edited and the French is now stale.

Adding new English content → translating it

The MT pipeline is **idempotent** — `translate_species` skips fields whose target column already has content and translates only what's empty. So the flow for new content is:

```
# 1. Edit description_en / ecology_notes_en / distribution_narrative_en /
#    morphology_en via Django admin, a content-fill management command, or
#    bulk import.

# 2. Run the MT pipeline (idempotent — only translates new/empty rows).
docker compose exec web python manage.py translate_species --locale fr
docker compose exec web python manage.py translate_species --locale de  # post-ABQ
docker compose exec web python manage.py translate_species --locale es  # post-ABQ

# 3. Review the new mt_draft rows in admin → writer_reviewed → human_approved.
#    See "Layer 2 — Species and Taxon content" above for the per-row flow.
```

Family-scoped run:

```
docker compose exec web python manage.py translate_species --locale fr --family
Bedotiidae
```

Dry-run (no DeepL calls, prints the job plan):

```
docker compose exec web python manage.py translate_species --locale fr --dry-run
```

Editing English on already-translated species

The `post_save` signal auto-demotes the locale rows back to `mt_draft` with a "needs re-review" note (the French text is preserved, but the review-gate serves the English fallback until somebody re-reviews). To **retranslate** rather than just re-review:

```
docker compose exec web python manage.py translate_species --locale fr --species 42
--force
```

Cron-friendly batch run (post-ABQ)

`translate_species` is safe to run on a schedule (idempotent + only translates empty cells). Once L4 ships and prod has the deployment, a weekly cron is a reasonable pattern: it queues new content for review without manual prompting.

Husbandry content (currently English-only)

`SpeciesHusbandry` is **not yet translatable** — see the post-L4 ticket to register it with `django-modeltranslation` and extend `translate_species` to walk husbandry fields too. Until that lands, husbandry pages render English on `/fr/`, `/de/`, `/es/` regardless of locale.

Quick reference

What needs fixing	Where	What advances it
UI button label, page heading, form field	`frontend/messages/<locale>.json`	PR review
Species long-form prose in French	Django admin → Translation statuses → row → `Target translation` field	Inline save
Status of a translation row	Translation statuses list → bulk-select → action dropdown	Admin actions: Advance / Approve / Send back
Glossary term convention used 5+ times	`.claude/agents/conservation-writer.md` (table) + sed-fix existing catalog/DB + bulk-demote affected rows	Re-review the demoted batch
English source content (any layer)	Edit en column or en.json	Auto-invalidates locale rows (Layer 2); doesn't touch Layer 1
New English content added (any species field)	Edit `<field>_en`, then `translate_species --locale <fr/de/es>`	Idempotent — translates only empty cells; produces `mt_draft` rows for review

---

Moving the local FR review work onto production (Path A)

The L3 review queue is being worked locally first because that's where the Bedotiidae batch was MT-translated and reviewed (April 2026). When prod / staging is deployable, the human-approved rows need to land there so the public site can serve them. Two-step move:

1. Dump the locale columns + TranslationStatus rows from local

```
# From the project root, with the local stack running:
docker compose exec -T db pg_dump \
    -U postgres -d mffcp \
    --data-only \
```

```
    --table=species_species \
    --table=i18n_translationstatus \
    --column-inserts \
    > /tmp/mffcp-fr-snapshot.sql
```

`--data-only` keeps the dump narrow (no schema). `--column-inserts` makes the rows easier to inspect / cherry-pick if needed. The file will include the English columns too — that's intentional; the same rows are the source-of-truth for both EN and FR/DE/ES.

2. Import on prod (or staging)

Two options depending on whether prod's species table is empty or has its own data:

If prod's species table is empty (fresh deploy):

```
# On the prod box, after psql access:
psql -U postgres -d mffcp_prod < mffcp-fr-snapshot.sql
```

If prod's species table is already populated (more likely):

The pg_dump above will conflict on primary keys. Cleanest path: write a small Python migration script that walks the dump, looks each species up by `scientific_name` (the natural key), and updates only the `<field>_<locale>` columns + creates the matching `i18n_translationstatus` rows. Save the script as `backend/i18n/management/commands/import_fr_snapshot.py` when implementing.

Either approach should run in a transaction and be re-runnable.

Caveats

- **Don't merge in human-approved rows blindly.** If prod has its own reviewed rows for the same species (e.g., a colleague approved a row through prod admin), the dump from local will overwrite. Prefer the field-level merge approach above, with a `--dry-run` flag that prints the would-update rows first.
- **Re-run signals on import.** The post_save signal handlers need to fire to keep things consistent. If using raw SQL load, remember to hit the species rows once after import (e.g., `Species.objects.all() .update(updated_at=timezone.now())`) to trigger a save. Or just use the field-level Python script which goes through the ORM.
- **TranslationStatus rows are coupled to a `content_type_id`.** The ContentType for Species must exist on the target DB before the dump is loaded. Standard Django setup handles this automatically.

When any of this gets implemented, that script and the actual runbook should land here as a section under "Path A — execution log."

Related

- Architecture: `docs/planning/architecture/i18n-architecture.md` — D5 (DeepL), §5 (signals), §4 (review-gate).
- Initiative hub: `docs/planning/i18n/README.md` — D1–D18 locked decisions.
- Auth handover (similar shape): `docs/handover/auth-gate-11-foundation.md` .
- Code:
  - `backend/i18n/management/commands/translate_species.py` — the MT pipeline.
  - `backend/i18n/signals.py` — auto-create / auto-invalidate logic.

- `backend/i18n/admin.py` — review surface (inline edit + actions).
- `backend/i18n/serializers.py` — `TranslationActualLocaleMixin` and the review gate.
- `frontend/scripts/translate-mt.mjs` — the catalog MT pipeline (Layer 1).
- `frontend/scripts/check-i18n-keys.mjs` — parity check.